Lab Assignment No.	01
Title	Design a distributed application using RPC for remote computation where client submits an integer value to the server and server calculates factorial and returns the result to the client program.
Roll No.	
Class	BE AI & DS
Date Of Completion	
Subject	Computer Laboratory III[417534]
Assessment Marks	
Assessor's Sign	

Assignment No. 01

Title: Design a Distributed Application Using RPC for Remote Computation of Factorial

Problem Statement: Develop a distributed application using Remote Procedure Call (RPC) where a client submits an integer value to a remote server. The server calculates the factorial of the given integer and returns the result to the client program. The application should demonstrate the concept of distributed computing by enabling remote execution of the factorial computation.

Prerequisites: Basic understanding of distributed systems, RPC mechanism, client-server architecture, factorial computation, programming in Python, and network communication fundamentals.

Software Requirements: Windows/Linux/MacOS, Python, RPC framework (XML-RPC, gRPC), IDE (VS Code, PyCharm), and networking tools.

Hardware Requirements: Intel Core i3 or higher, 4GB RAM, 100MB free disk space, and network connectivity for client-server communication.

Theory:

Introduction to Remote Procedure Call (RPC):

Remote Procedure Call (RPC) is a communication protocol that allows a program to execute procedures on a remote server as if they were local function calls. The RPC mechanism abstracts network communication complexities, making distributed computing seamless and efficient.

Working of RPC:

1. The client sends a request to the server with the necessary input data.
2. The server processes the request, executes the remote function (factorial calculation in this case), and prepares the response.
3. The server sends the computed result back to the client.
4. The client receives the result and processes it accordingly.

Factorial Computation:

The factorial of a non-negative integer **n** is the product of all positive integers from **1 to n** and is denoted as **n!**. It is defined as:

For example:

- 5! = 5 × 4 × 3 × 2 × 1 = 120
- 3! = 3 × 2 × 1 = 6
- 1! = 1
- 0! = 1 (by definition)

Implementation Approach:

To implement an RPC-based distributed factorial computation system, the following approach is used:

1. **Client Program:** Sends an integer to the server via an RPC request.
2. **Server Program:** Receives the request, computes the factorial of the given integer, and sends the result back to the client.
3. **Communication Layer:** Handles the exchange of requests and responses between the client and server using RPC mechanisms.

Advantages of RPC-Based Computation:

- Simplifies distributed computing by making remote function calls appear as local calls.
- Reduces computational load on client devices by offloading heavy calculations to the server.
- Enables modular and scalable application design.
- Facilitates resource sharing and efficient computing in networked environments.

Source Code –

Client.py

```
import xmlrpc.client

# Create an XML-RPC client
with xmlrpc.client.ServerProxy("http://localhost:8000/RPC2") as proxy:
    try:
        # Prompt the user to enter a number
        input_value_str = input("Enter the number: ")
        input_value = int(input_value_str)

        # Call the server's calculate_factorial method
        result = proxy.calculate_factorial(input_value)
        print(f"Factorial of {input_value} is: {result}")
    except Exception as e:
        print(f"Error: {e}")
```

Output -

Enter the number: 12

Factorial of 12 is: 479001600

Server.py

```
from xmlrpc.server import SimpleXMLRPCServer
from xmlrpc.server import SimpleXMLRPCRequestHandler

class FactorialServer:
    def calculate_factorial(self, n):
        if n < 0:
            raise ValueError("Input must be a non-negative integer.")
        result = 1
        for i in range(1, n + 1):
            result *= i
        return result

# Restrict to a particular path
class RequestHandler(SimpleXMLRPCRequestHandler):
    rpc_paths = ('/RPC2',)

# Create server
with SimpleXMLRPCServer(('localhost', 8000), requestHandler=RequestHandler) as
server:
    server.register_introspection_functions()
    server.register_instance(FactorialServer())
    print("FactorialServer is ready to accept requests.")
    server.serve_forever()
```

Output - FactorialServer is ready to accept requests.

Conclusion: This practical demonstrates the efficiency of RPC in distributed computing by remotely executing factorial calculations. It highlights modular design, computational offloading, and seamless communication in client-server architectures.